

AI Refactoring Report

Project: Vehicle Rental Management System (Phase 1)

Scope: Refactoring-only assistance applied to already-working, test-covered code.

Safety net: The full JUnit 5 suite (33 tests) was run before and after every change. All 33 tests pass after refactoring, so each change is behavior-preserving.

Note on method: AI was used strictly for *refactoring* — improving the shape of existing code without changing what it does. Every “before” snapshot below is the real prior state of the file, and every “after” snapshot is the committed result. No behavior was added or removed; the passing tests are the proof.

1. Files refactored with AI

#	File	Smell addressed	Technique
1	notification/EmailReminderObserver.java	Long Method; nested ternary inside string concatenation; repeated getVehicle() calls	Extract Method
2	RentalServiceTest, DurationLimitRuleTest, EmailReminderObserverTest (+ new TestFixtures)	Duplicated Code — the same Vehicle constructor call with magic values repeated across test classes	Extract Class / Factory Method

2. Prompts used

Prompt for refactoring 1:

“This onRentalExpiring method builds an e-mail subject and body in one long block and decides the wording with a nested ternary inside the string. Refactor it for readability using Extract Method, without changing the produced text. Keep it behavior-preserving so the existing unit tests keep passing.”

Prompt for refactoring 2:

“Several test classes construct the exact same Vehicle (V-1 / NAB-1234, Toyota Corolla, 35.00) inline. Remove this duplication by extracting a shared test factory, and update the tests to use it. Do not change any assertions.”

3. & 4. Before / After

Refactoring 1 — EmailReminderObserver

Before:

```
@Override
public void onRentalExpiring(Rental rental, long daysUntilExpiry) {
    String subject = "Rental expiry reminder - " + rental.getVehicle().getPlateNumber();
    String body = "Dear " + rental.getCustomerName() + ",\n"
        + "Your rental of " + rental.getVehicle().getBrand() + " "
        + rental.getVehicle().getModel()
        + " (plate " + rental.getVehicle().getPlateNumber() + ") "
        + "ends on " + rental.getEndDate() + " ("
        + (daysUntilExpiry == 0
            ? "today"
            : daysUntilExpiry + " day(s) left")
        + ").\n"
        + "Please return the vehicle on time to avoid late penalties.";
    emailService.sendEmail(rental.getCustomerEmail(), subject, body);
}
```

After:

```
@Override
public void onRentalExpiring(Rental rental, long daysUntilExpiry) {
    emailService.sendEmail(
        rental.getCustomerEmail(),
        buildSubject(rental),
        buildBody(rental, daysUntilExpiry));
}

private String buildSubject(Rental rental) {
    return "Rental expiry reminder - " + rental.getVehicle().getPlateNumber();
}

private String buildBody(Rental rental, long daysUntilExpiry) {
    Vehicle vehicle = rental.getVehicle();
    return "Dear " + rental.getCustomerName() + ",\n"
        + "Your rental of " + vehicle.getBrand() + " " + vehicle.getModel()
        + " (plate " + vehicle.getPlateNumber() + ")\n"
        + "ends on " + rental.getEndDate()
        + " (" + describeRemaining(daysUntilExpiry) + ").\n"
        + "Please return the vehicle on time to avoid late penalties.";
}

private String describeRemaining(long daysUntilExpiry) {
    if (daysUntilExpiry == 0) {
        return "today";
    }
    return daysUntilExpiry + " day(s) left";
}
```

What improved: the public method now reads as three intent-revealing steps (subject, body, send). The remaining-time wording moved out of the string into a named `describeRemaining` method, replacing the nested ternary. `rental.getVehicle()` is resolved once into a local variable instead of being called three times.

Refactoring 2 — Duplicated test vehicle

Before (repeated verbatim across several test classes):

```
Vehicle vehicle = new Vehicle("V-1", "NAB-1234", "Toyota", "Corolla",
    new BigDecimal("35.00"), VehicleStatus.AVAILABLE);
```

After — a single shared factory in `TestFixtures.java`:

```
public final class TestFixtures {

    private TestFixtures() {}

    public static Vehicle corolla(String id, String plate, VehicleStatus status) {
        return new Vehicle(id, plate, "Toyota", "Corolla",
            new BigDecimal("35.00"), status);
    }

    public static Vehicle availableCorolla() {
        return corolla("V-1", "NAB-1234", VehicleStatus.AVAILABLE);
    }
}
```

Call sites become one readable line, e.g.

```
vehicleRepository.save(TestFixtures.availableCorolla());
```

What improved: the magic values live in one place; a change to the demo vehicle now touches a single file, and the tests read at a higher level of intent. Unused imports (`BigDecimal`, `VehicleStatus`, `Vehicle`)

were removed from the call sites as a side effect.

5. Accepted vs rejected suggestions

Suggestion	Decision	Reason
Extract buildSubject / buildBody / describeRemaining (refactoring 1)	Accepted	Splits one long method into small, named units; directly removes the nested ternary. Tests confirm the produced text is unchanged.
Resolve rental.getVehicle() into a local variable	Accepted	Removes three repeated calls and reads better; no behavioral effect.
Rewrite the e-mail body as a Java text block with String.format	Rejected	The body mixes several interpolated fields with the dynamic today / N day(s) left phrase. A text block would force positional String.format placeholders, which is more error-prone and risks changing the exact output the unit test asserts on.
Extract a shared TestFixtures factory (refactoring 2)	Accepted	Removes real duplication of the vehicle constructor and its magic values across test classes.
Build a full Builder pattern for test vehicles	Rejected	Over-engineering (YAGNI) for the current tests. Two small factory methods cover every existing need; a builder would add ceremony without a caller that benefits.

Verification

```
mvnw test -> Tests run: 33, Failures: 0, Errors: 0, Skipped: 0 -> BUILD SUCCESS
```

Because the same 33 tests pass before and after, the refactorings are behavior-preserving: the code is cleaner, and what it does is unchanged.